

Softwareentwicklung ohne Abhängigkeiten

Stefan Lieser

stefan@lieser-online.de



<http://refactoring-legacy-code.net>

MIT

Flow Design

ZU

CLEAN CODE

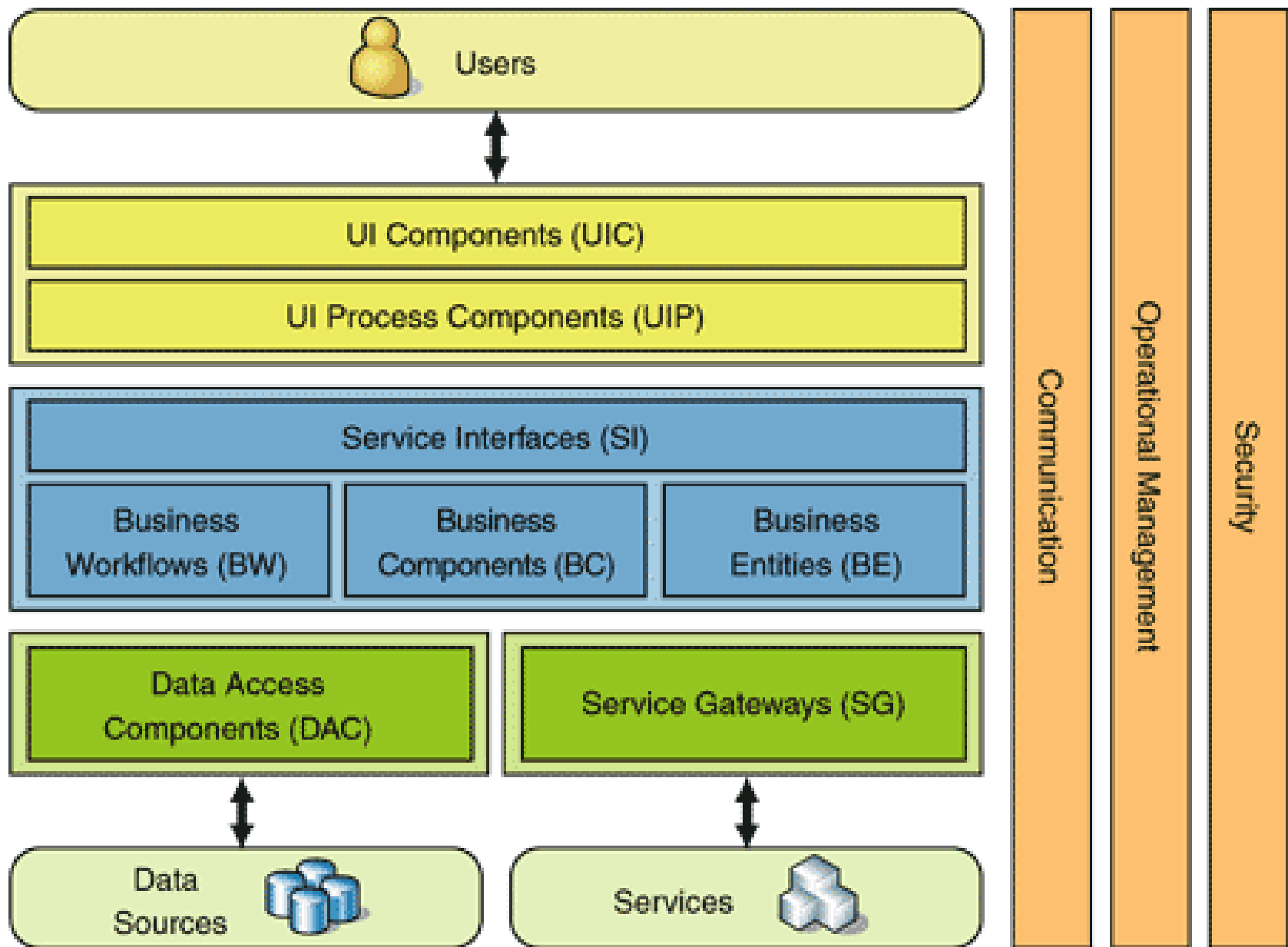
STEFAN LIESER



<https://flow-design.info/buch>

Vorabveröffentlichung

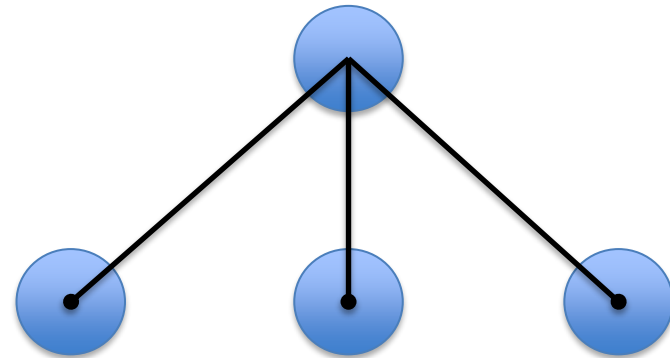
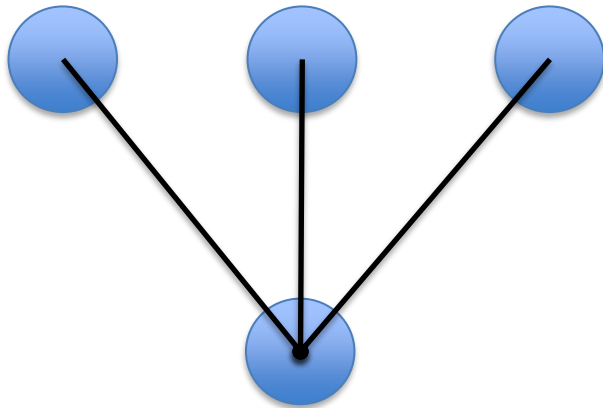
- E-Book (PDF, mobi, epub)
- Inhalt vollständig (400 Seiten)
- Endgültige Version im Preis enthalten
- Sonderpreis 19,95 EUR



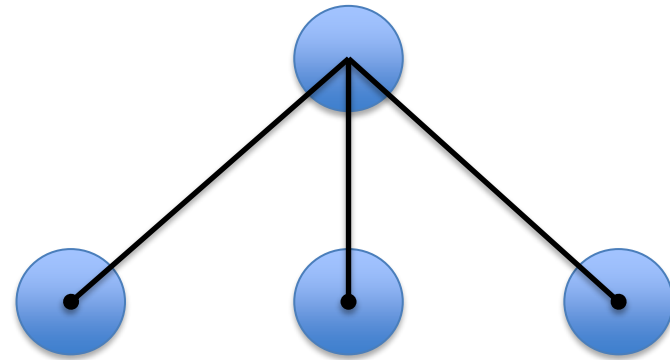
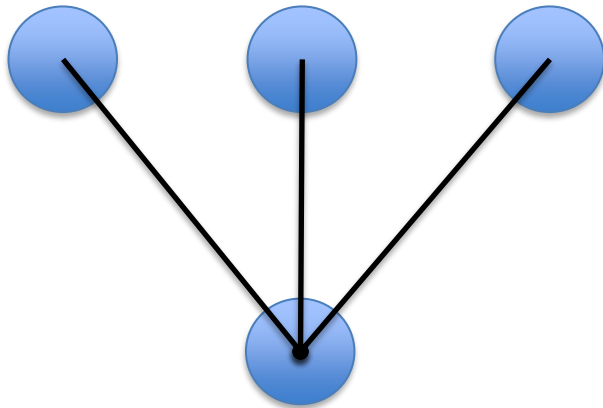
Korrektheit

Korrektheit & Wandelbarkeit

Typische Struktur von Abhängigkeiten



Typische Struktur von Abhängigkeiten

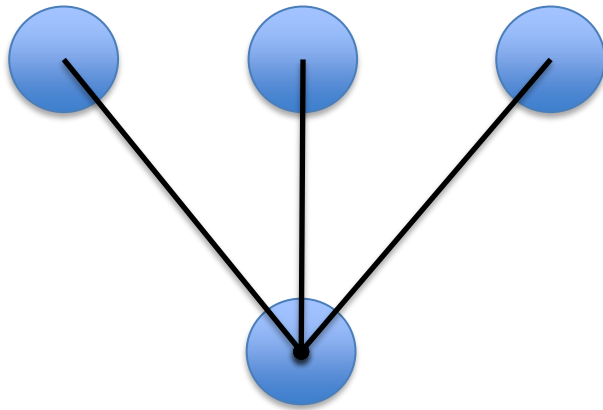


= Funktionseinheit (Methode, Klasse, etc.)

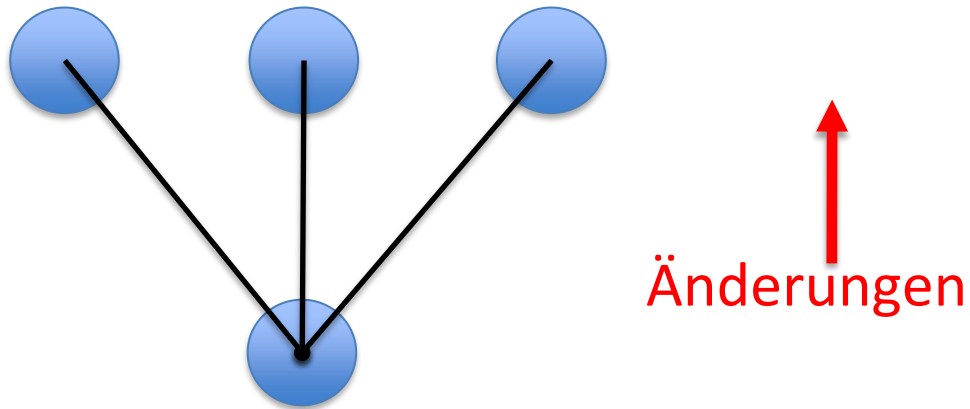
A — B

= Abhängigkeit (A ist abhängig von B)

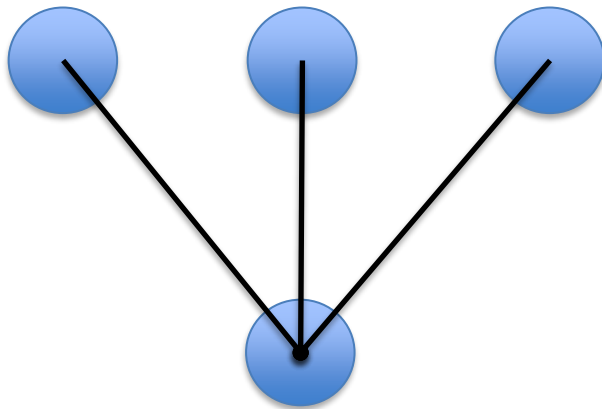
Typische Struktur von Abhängigkeiten



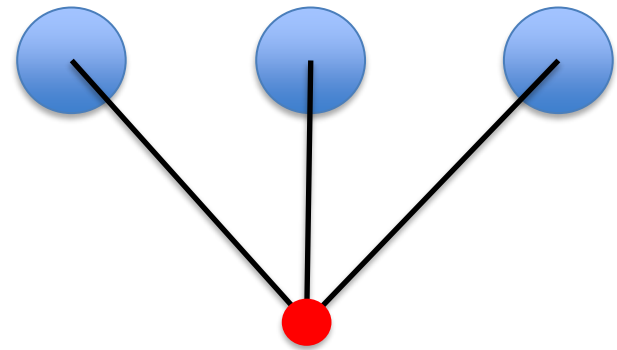
Typische Struktur von Abhängigkeiten



Typische Struktur von Abhängigkeiten



=>

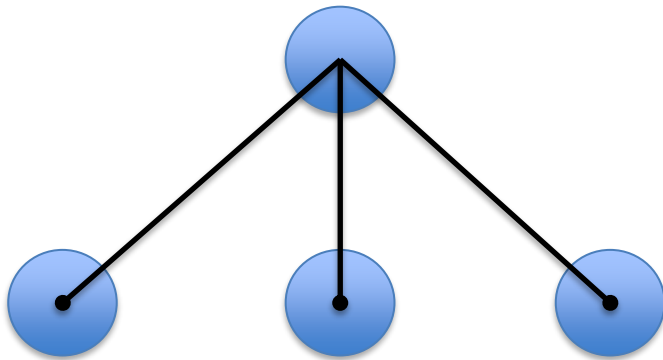


Keine Domänenlogik,
nur Daten

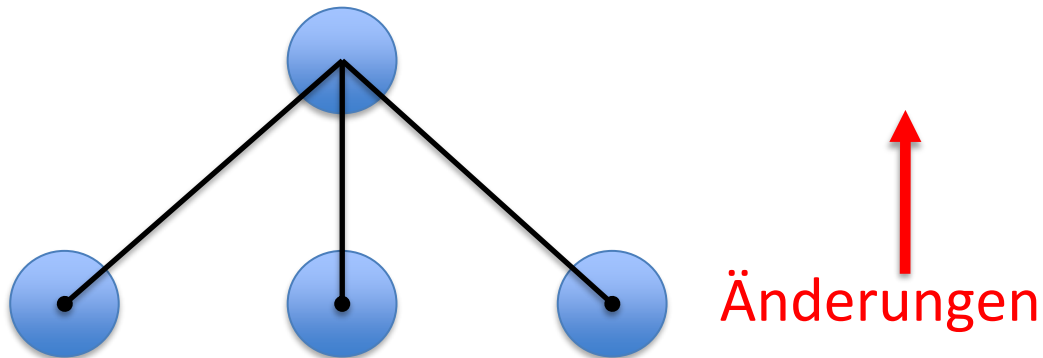
Domänenlogik

Logik, die das Thema
der Anwendung betrifft.

Typische Struktur von Abhängigkeiten

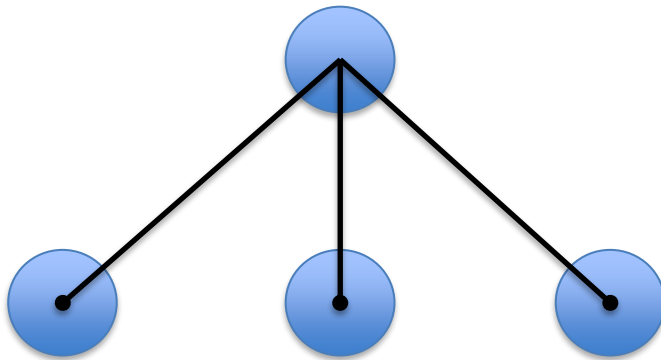


Typische Struktur von Abhängigkeiten

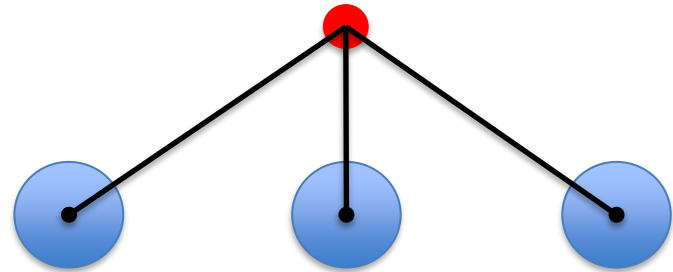


Typische Struktur von Abhängigkeiten

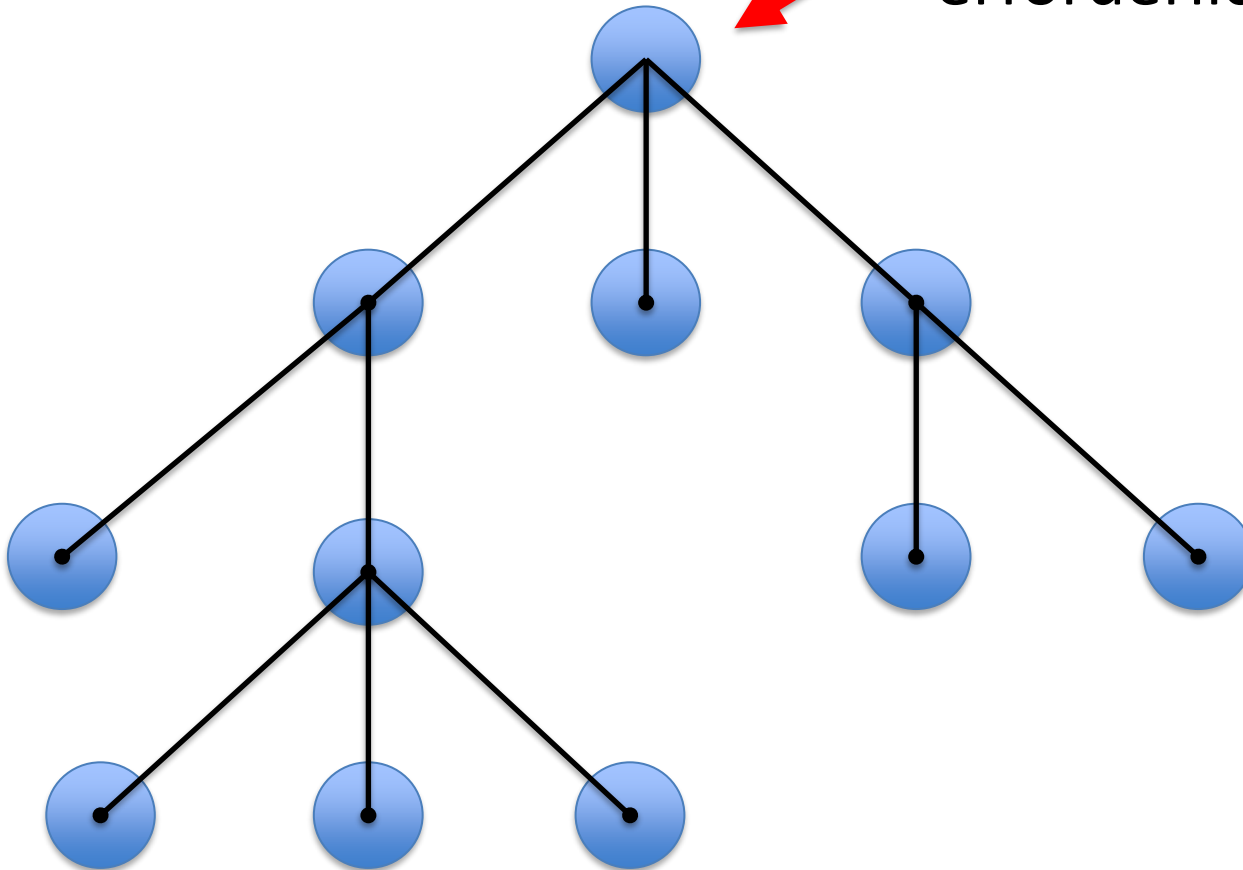
Keine Domänenlogik,
nur Integration

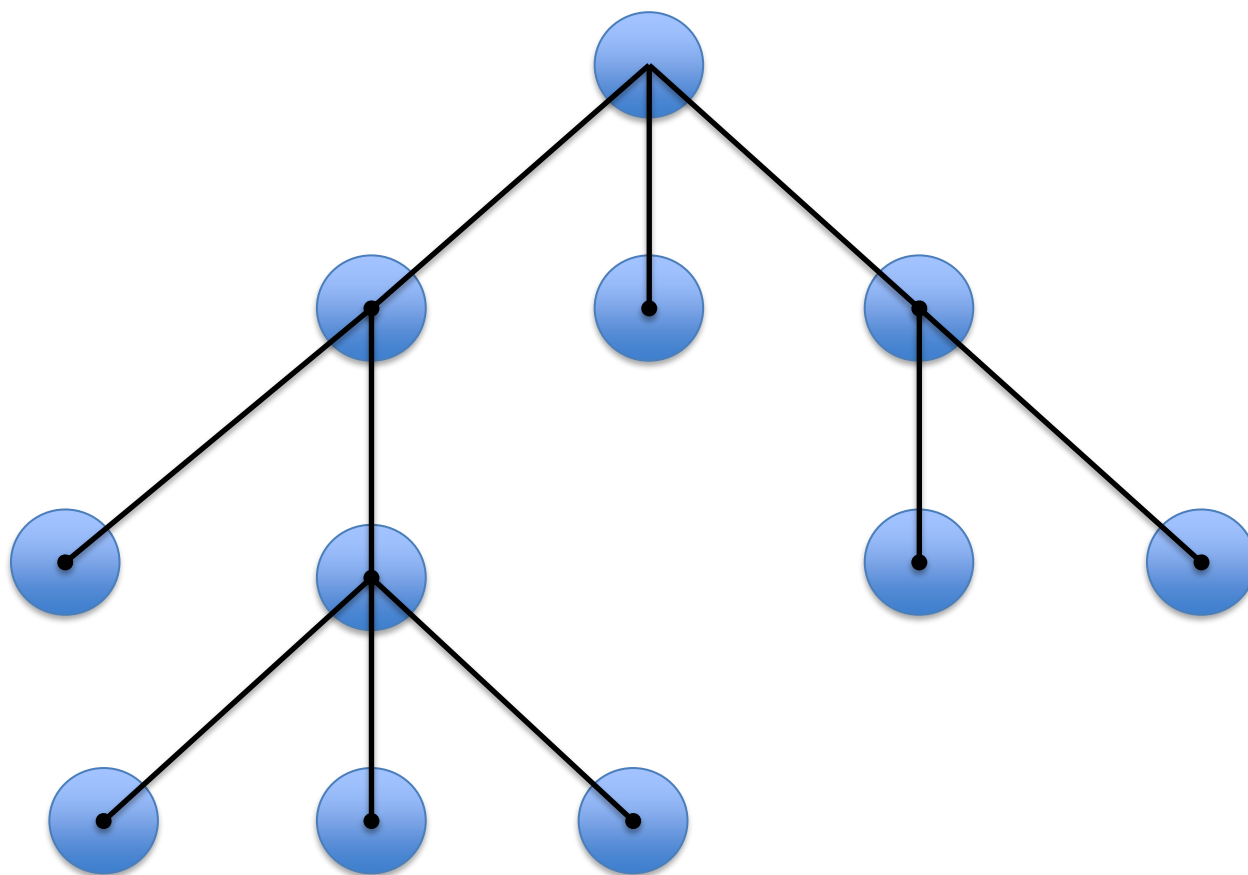


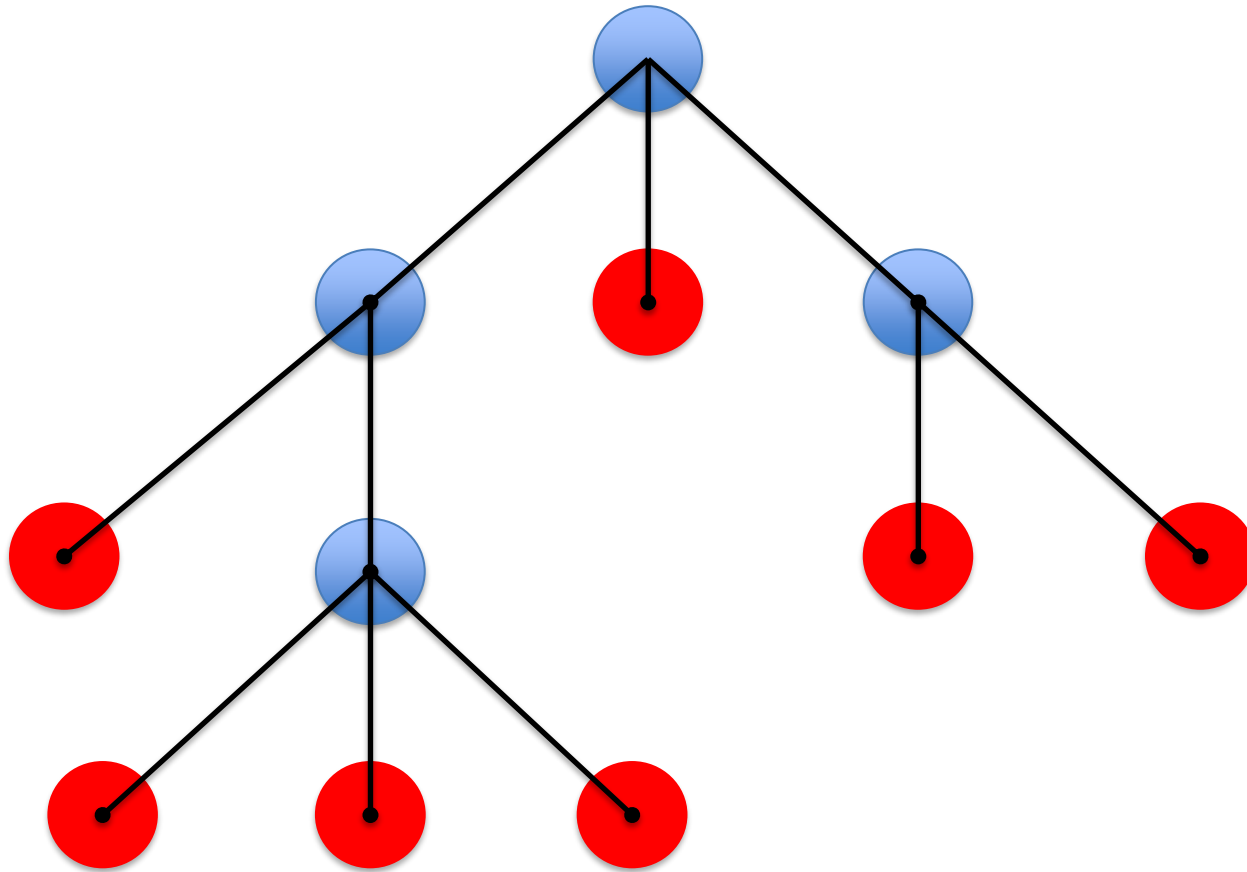
$\Rightarrow$



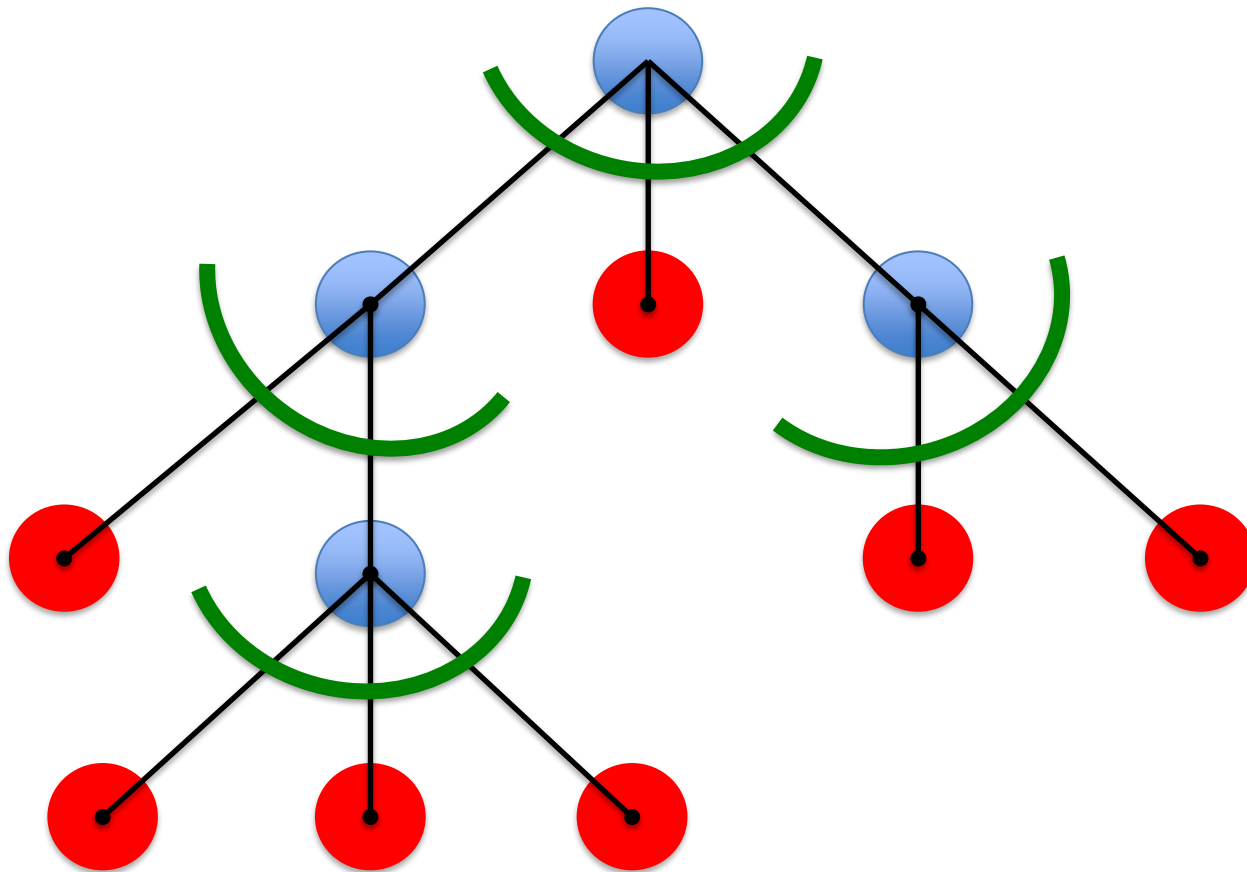
Integrationstests
erforderlich!!!



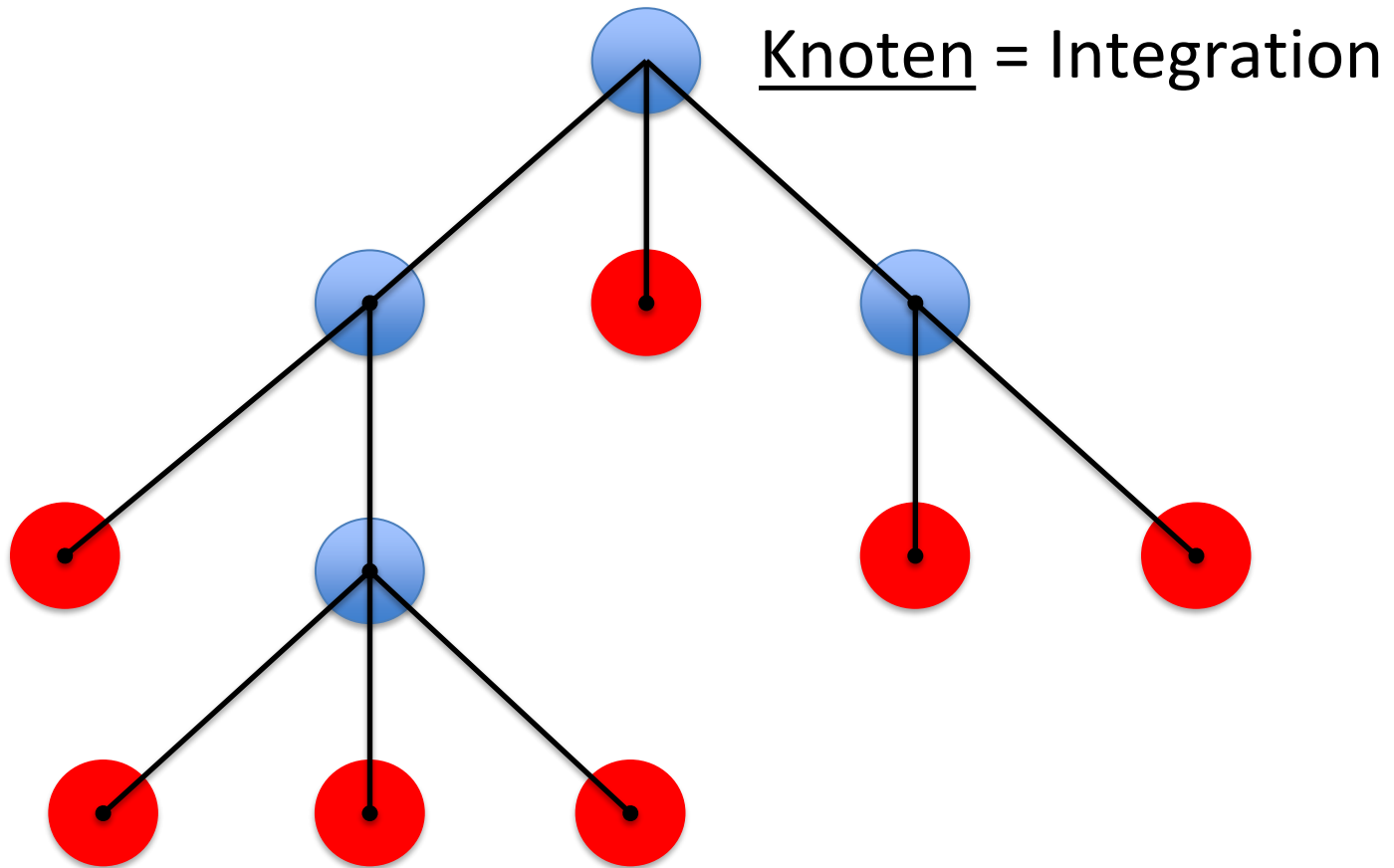




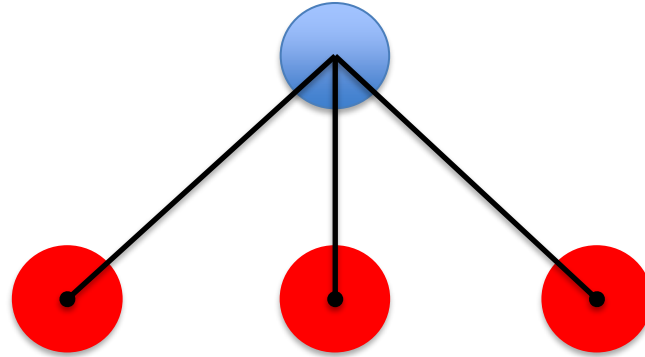
● Unit Tests einfach, da keine Abhängigkeiten



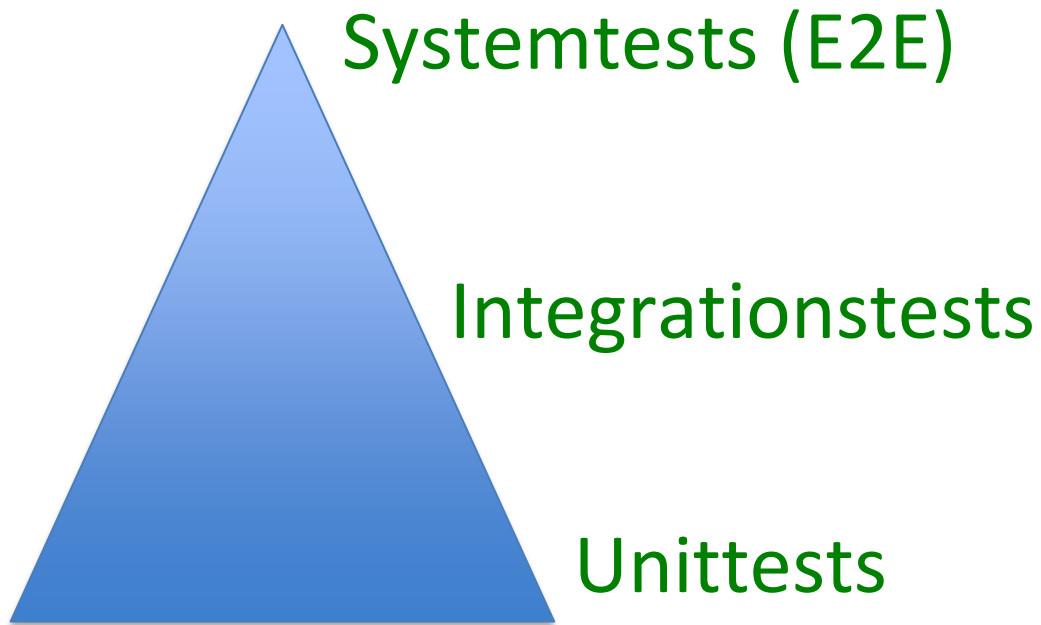
● Unit Tests aufwändig, Einsatz von Attrappen

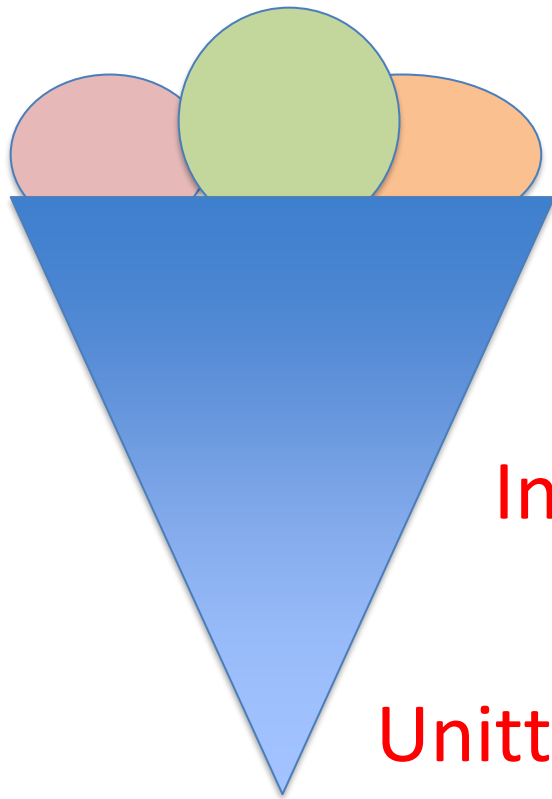


Blätter = Operationen = Domänenlogik



Integration Operation
Segregation Principle (IOSP)

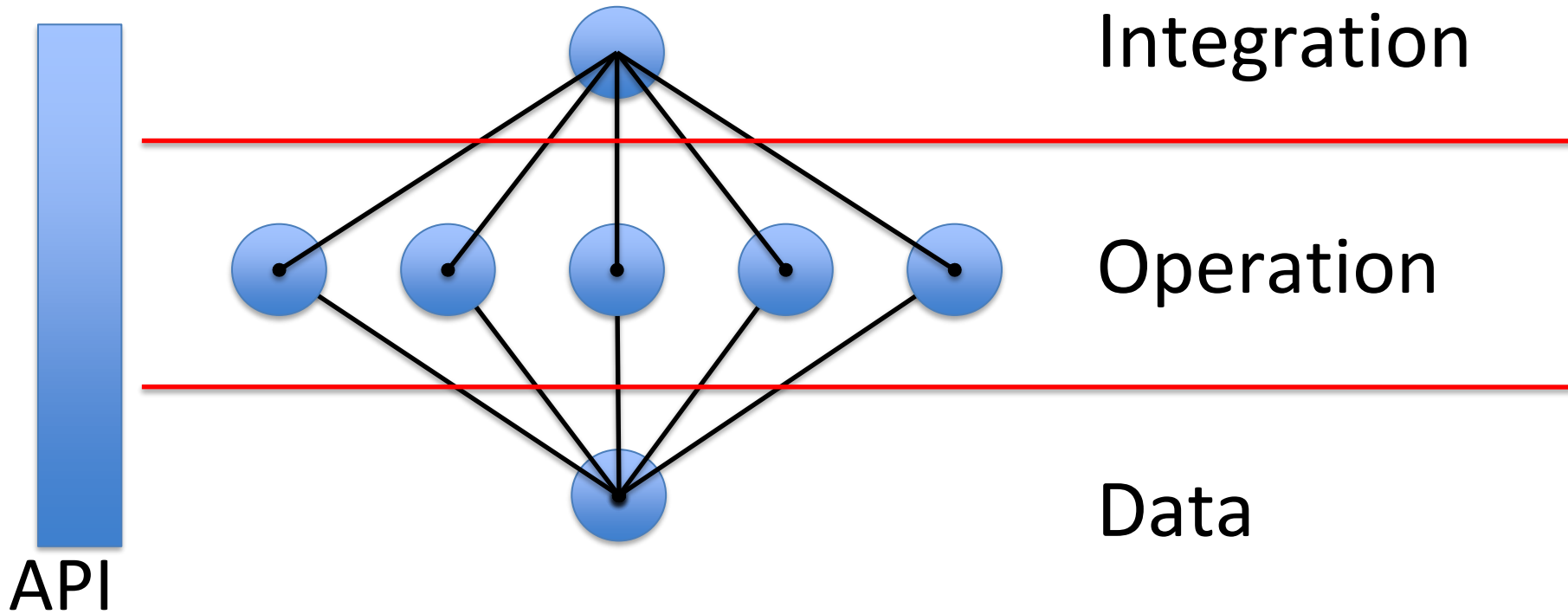




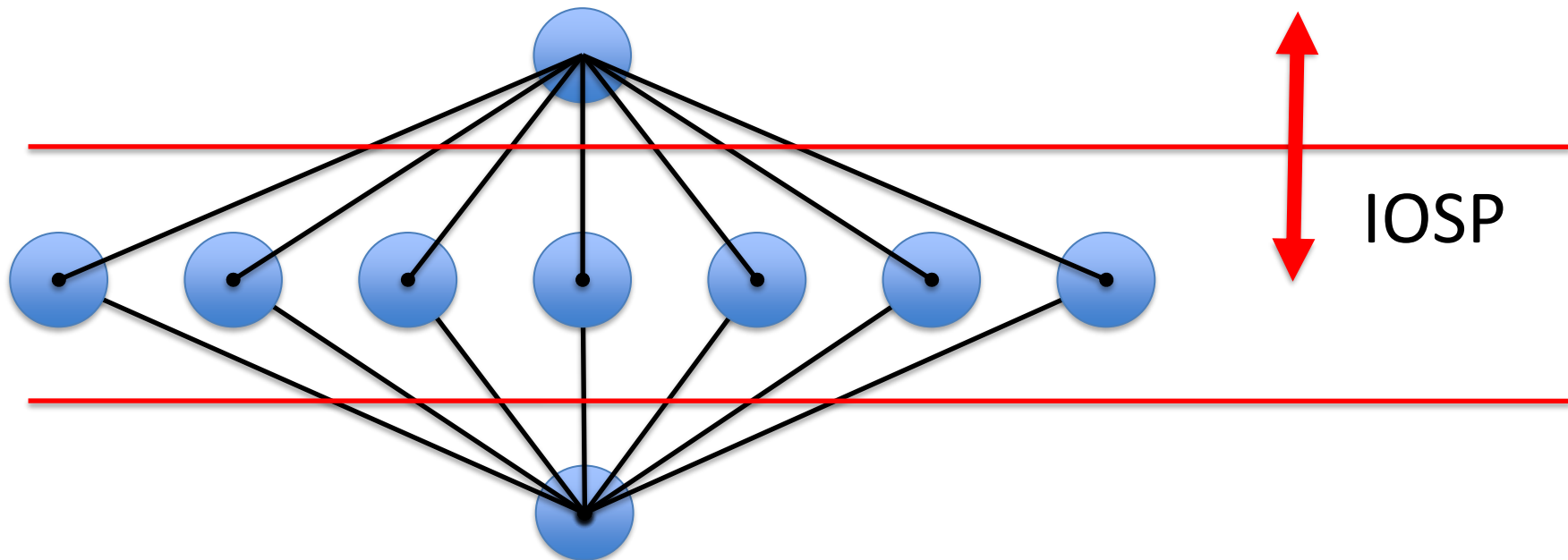
Systemtests (E2E)

Integrationstests

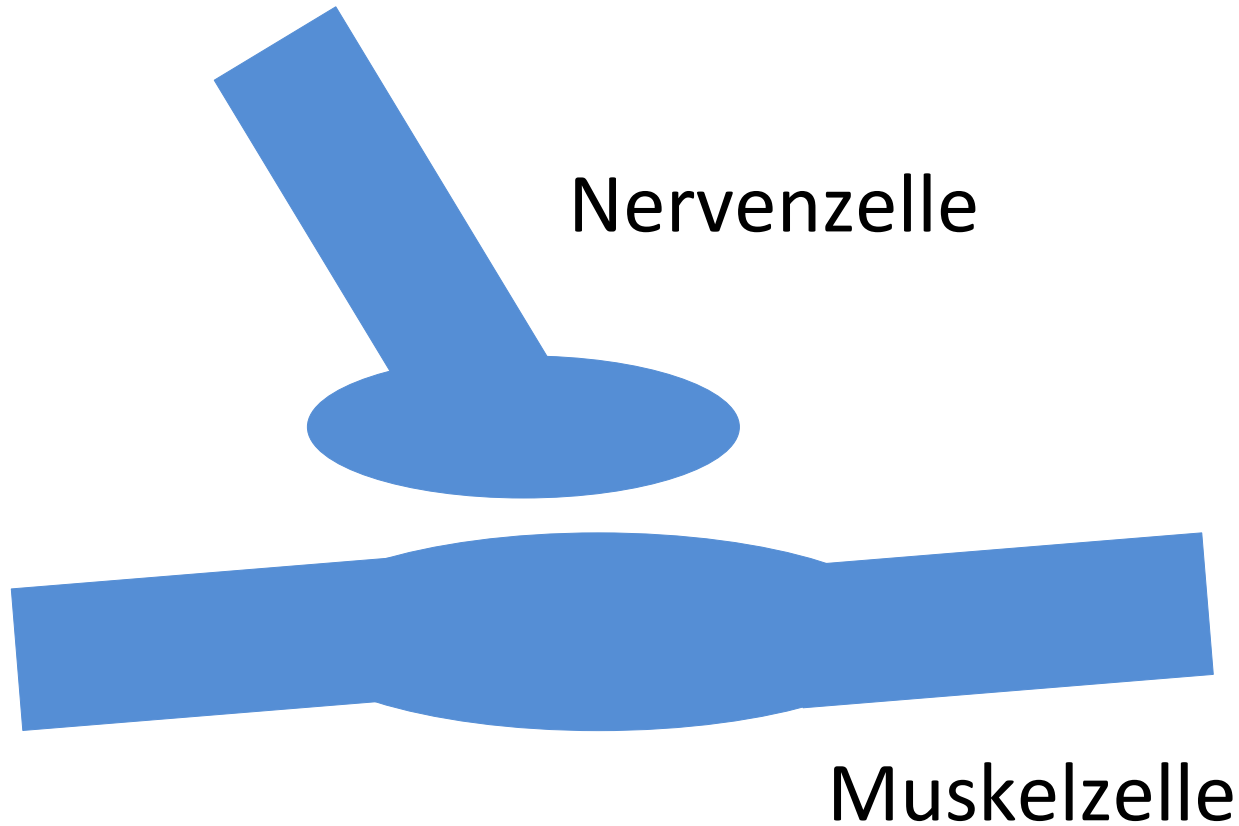
Unittests



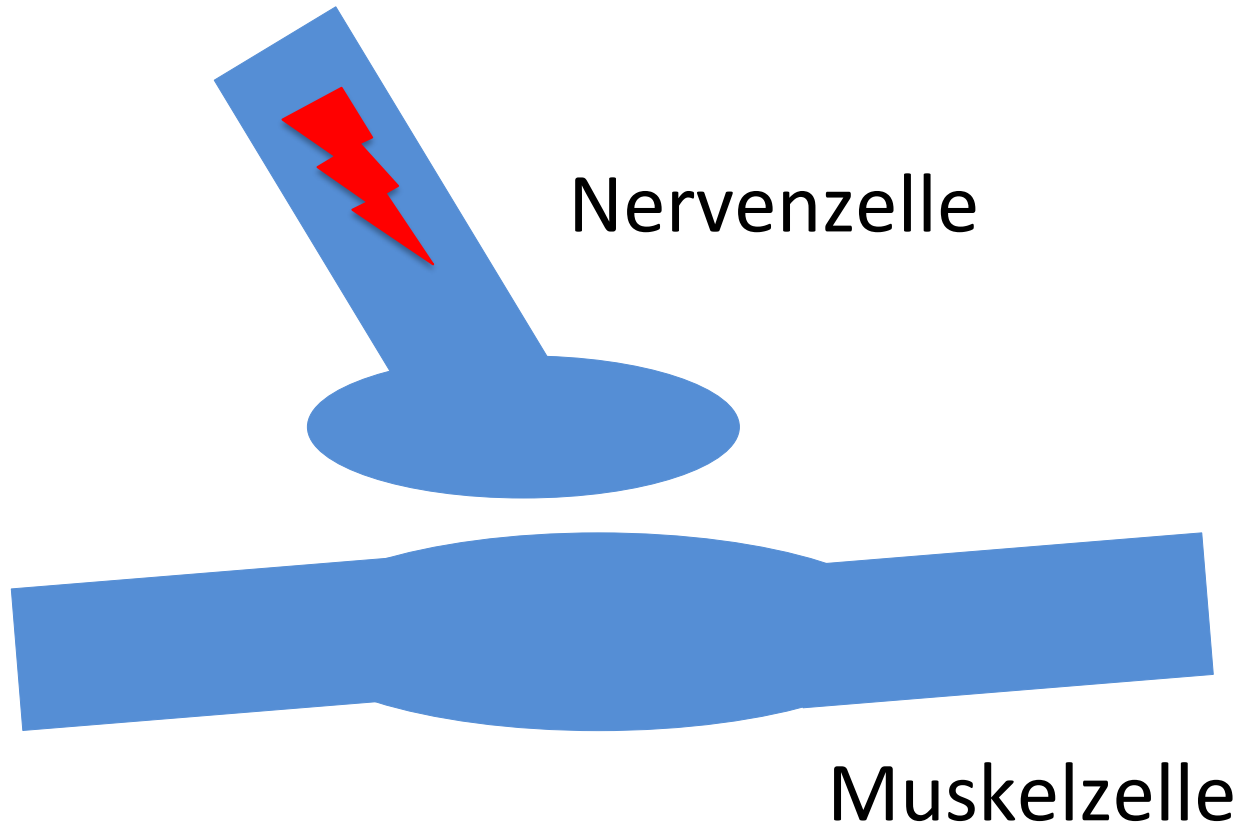
Integration Operation
Data API (IODA)



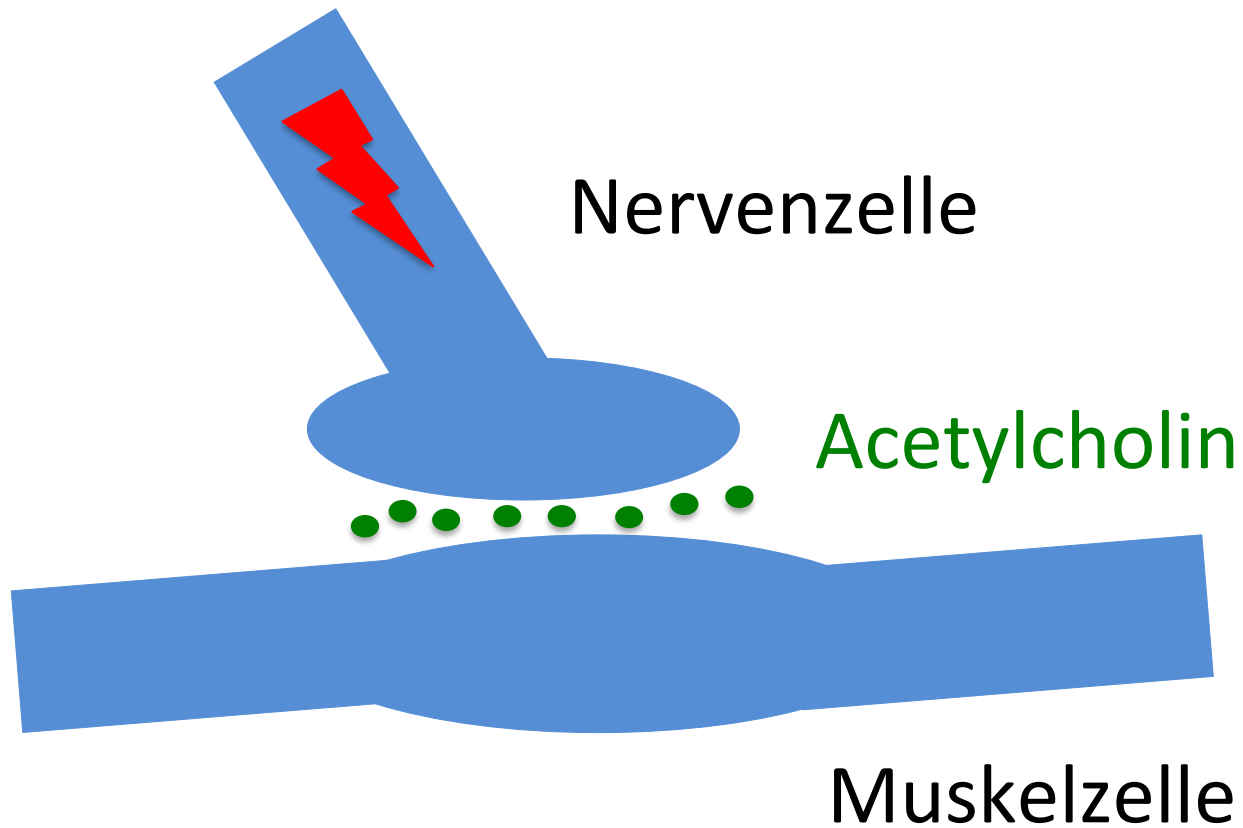
Motorische Endplatte



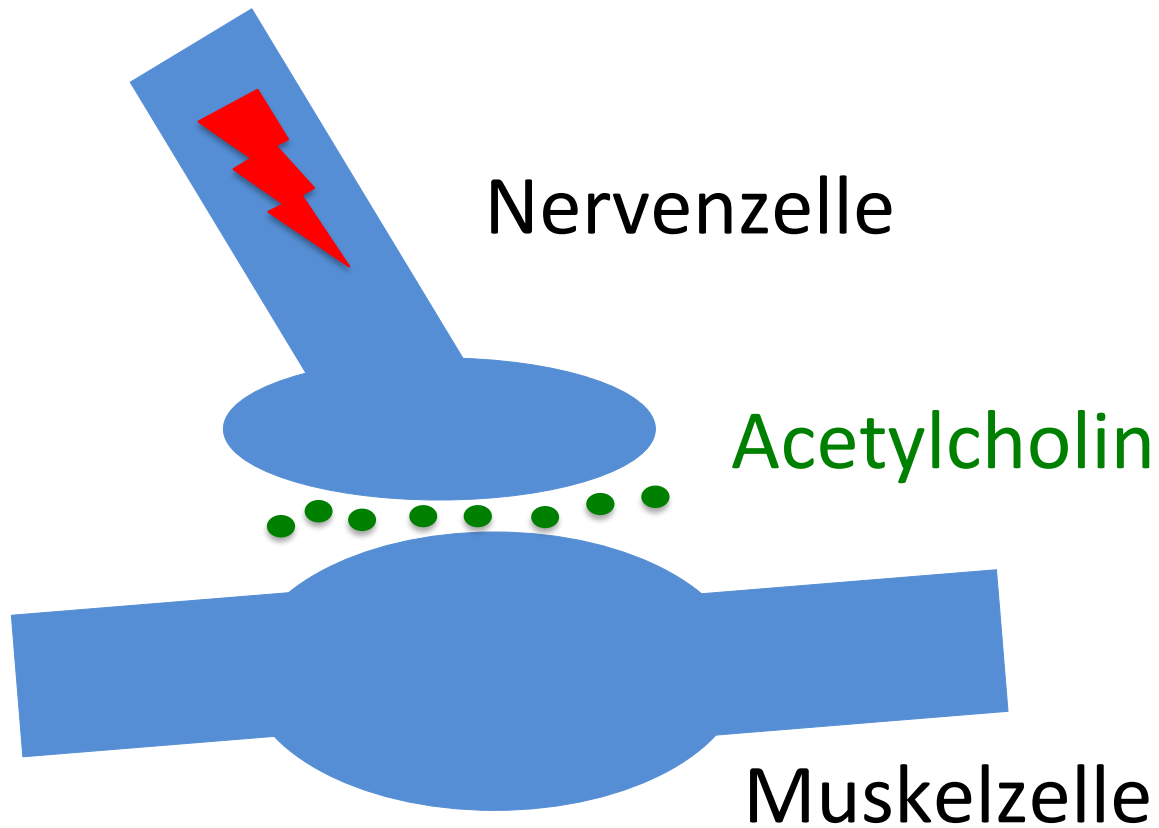
Motorische Endplatte



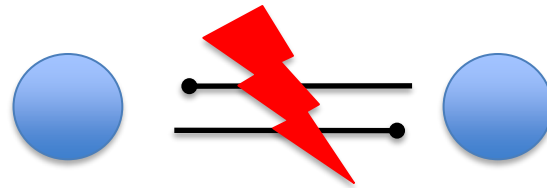
Motorische Endplatte



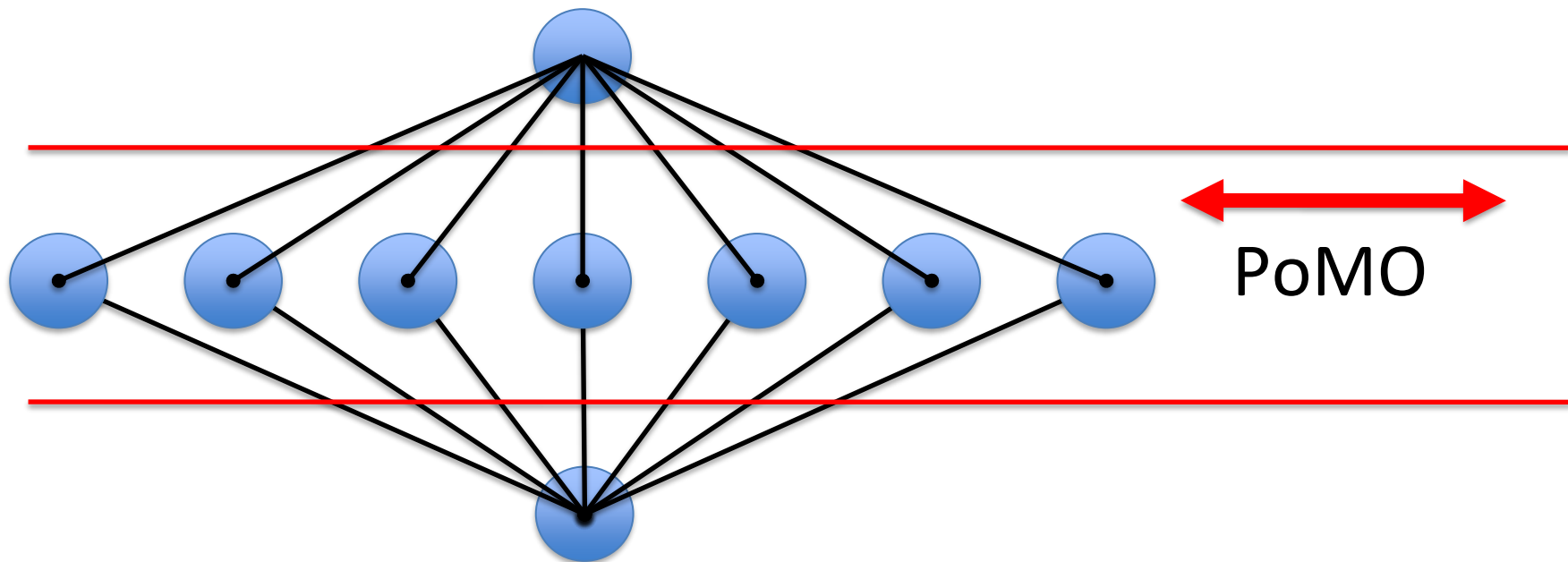
Motorische Endplatte

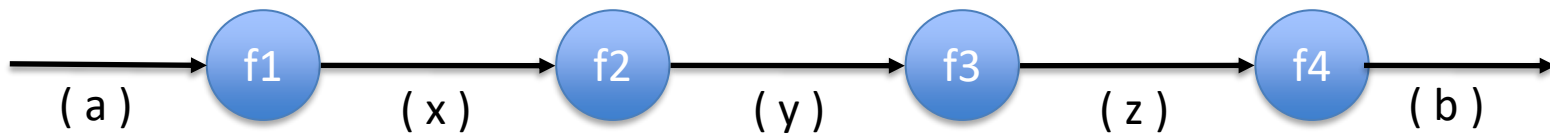


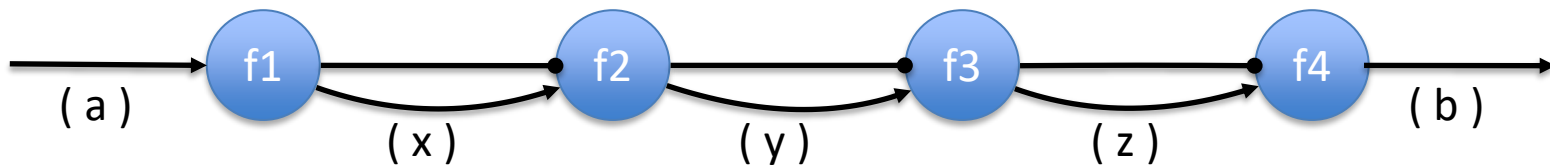
Principle of Mutual Oblivion (PoMO)



Prinzip der
gegenseitigen
Nichtbeachtung



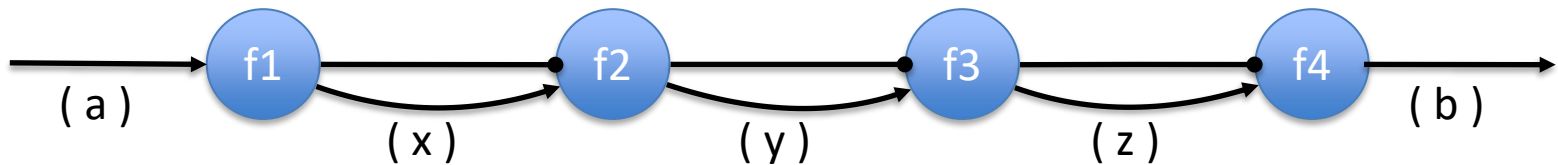




PoMO verletzt

(Principle of Mutual Oblivion)

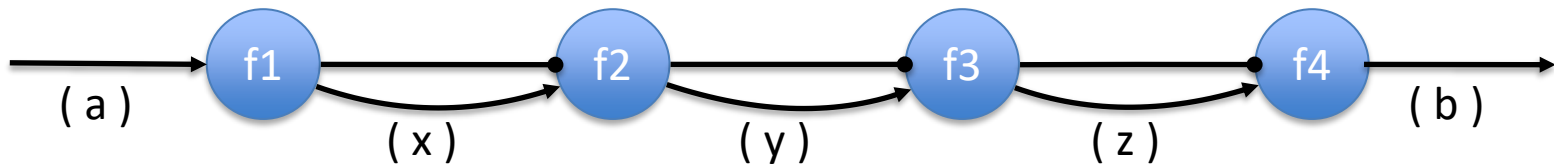
-> f1 ist abhängig von f2

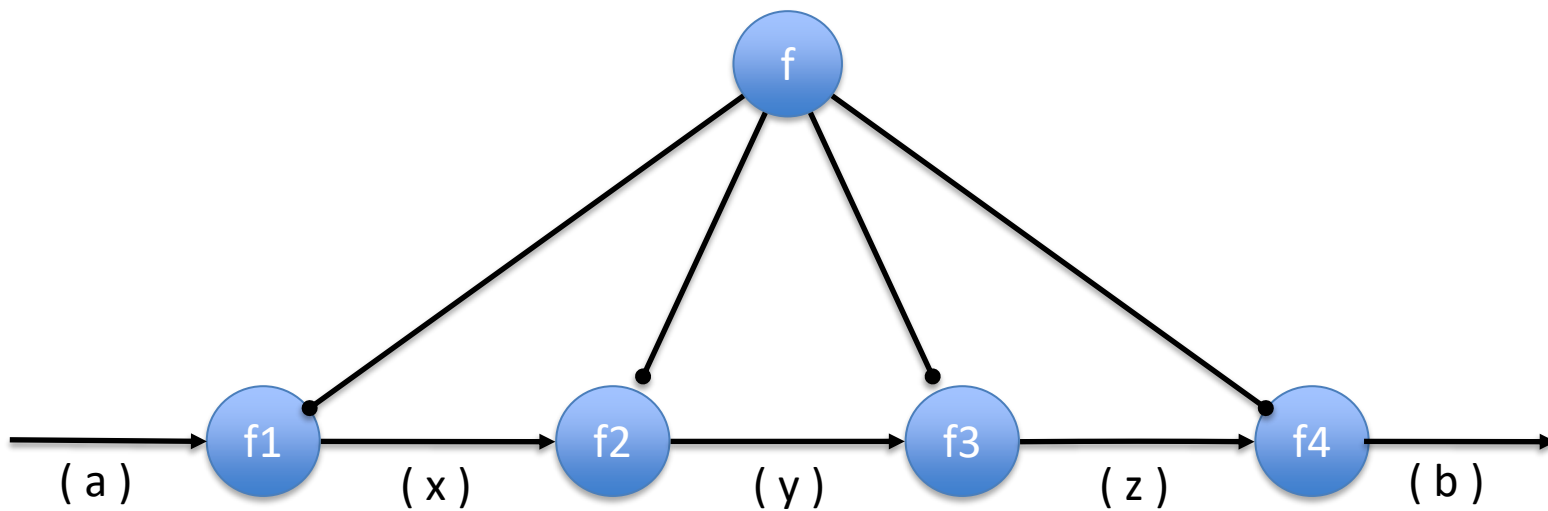


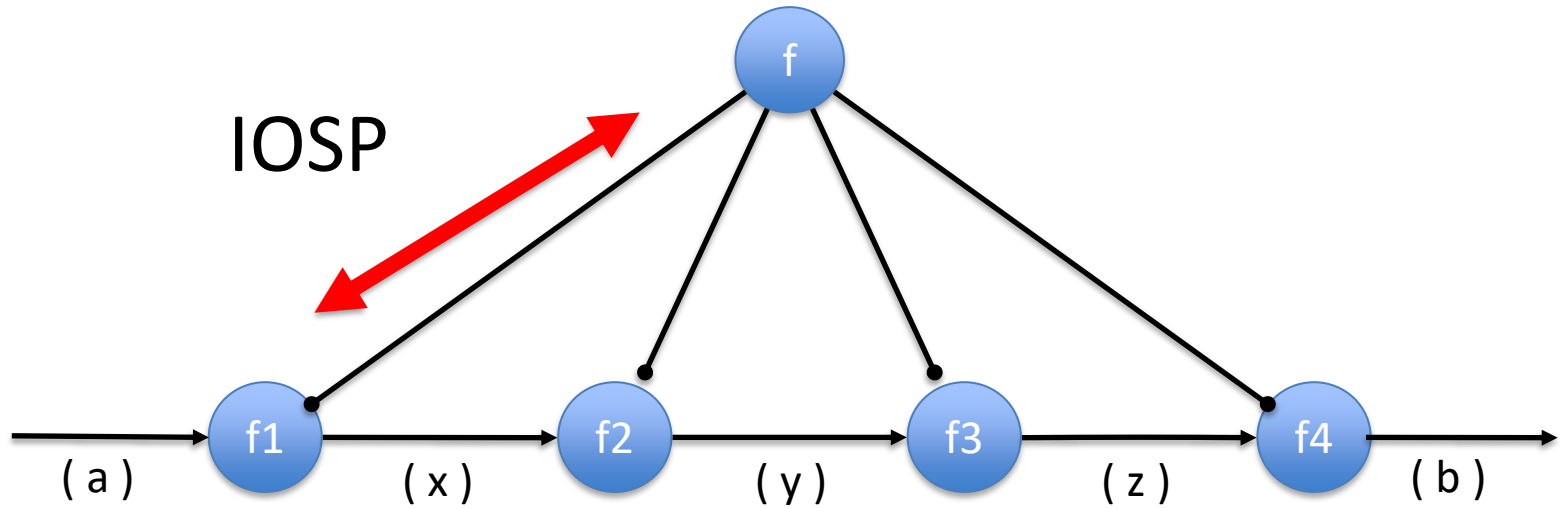
IOSP verletzt

(Integration Operation Segregation Principle)

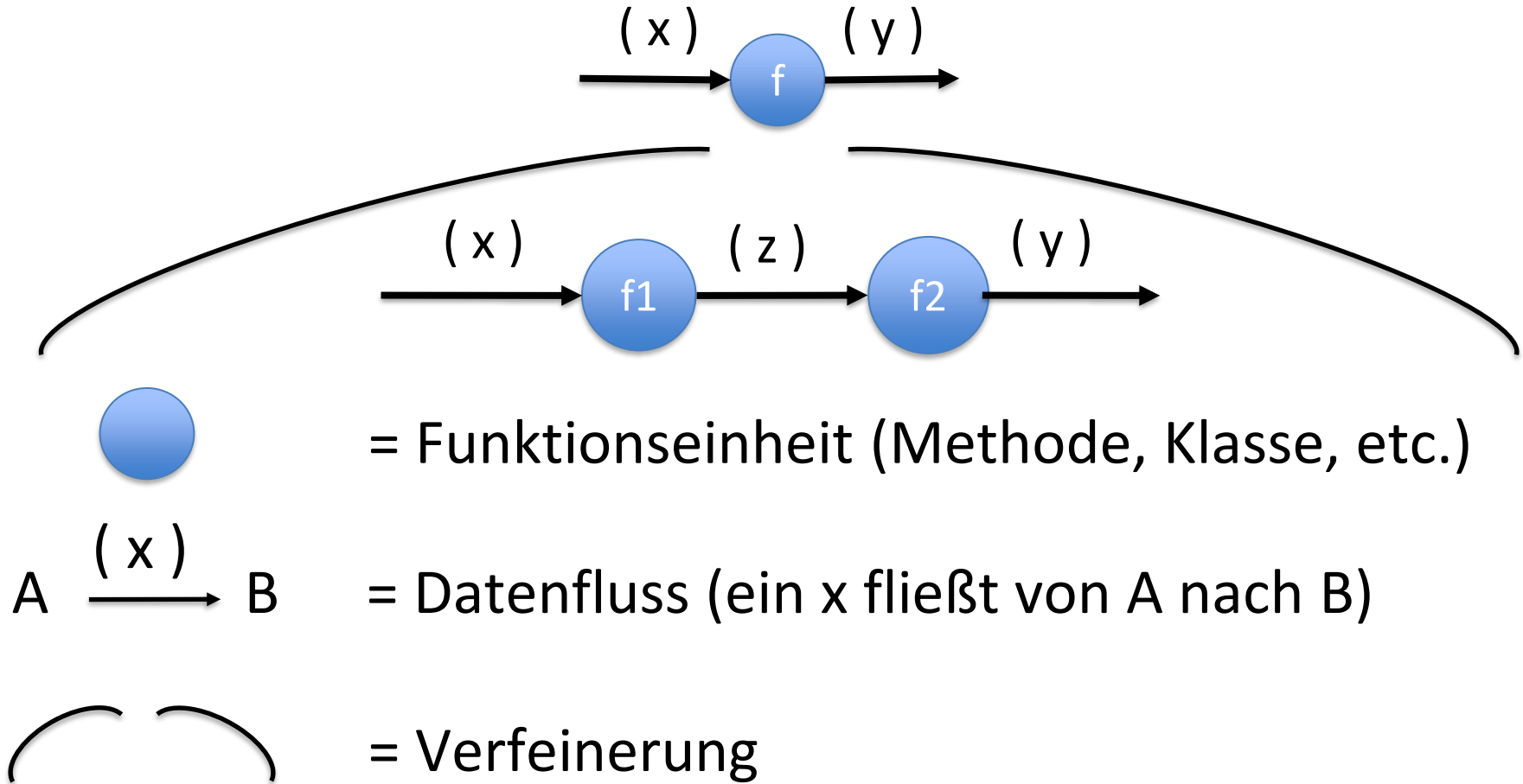
-> f1 enthält Domänenlogik und integriert f2



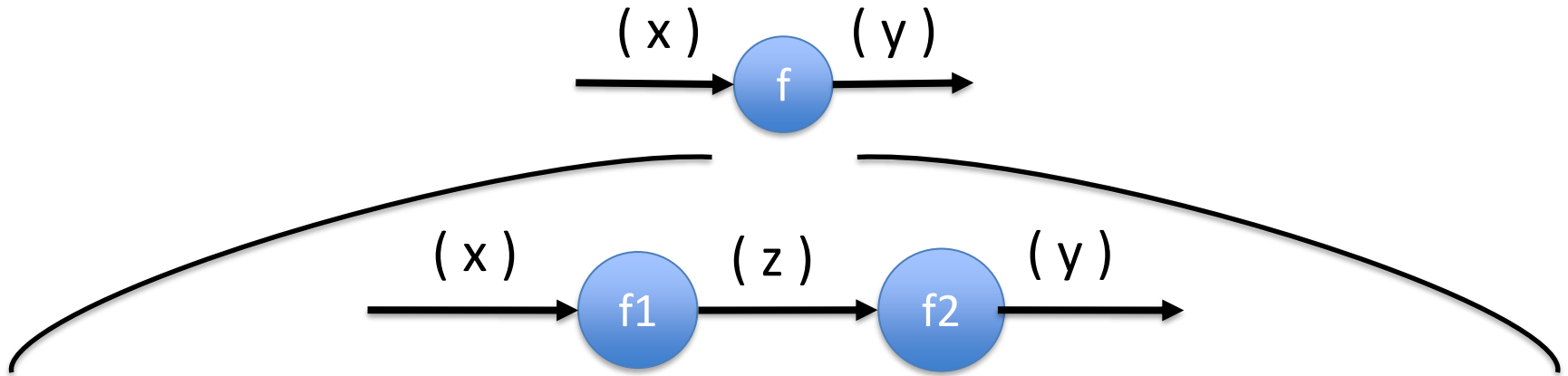




Flow Design: Datenflussdiagramme



Mit Abhängigkeiten (FALSCH!!)

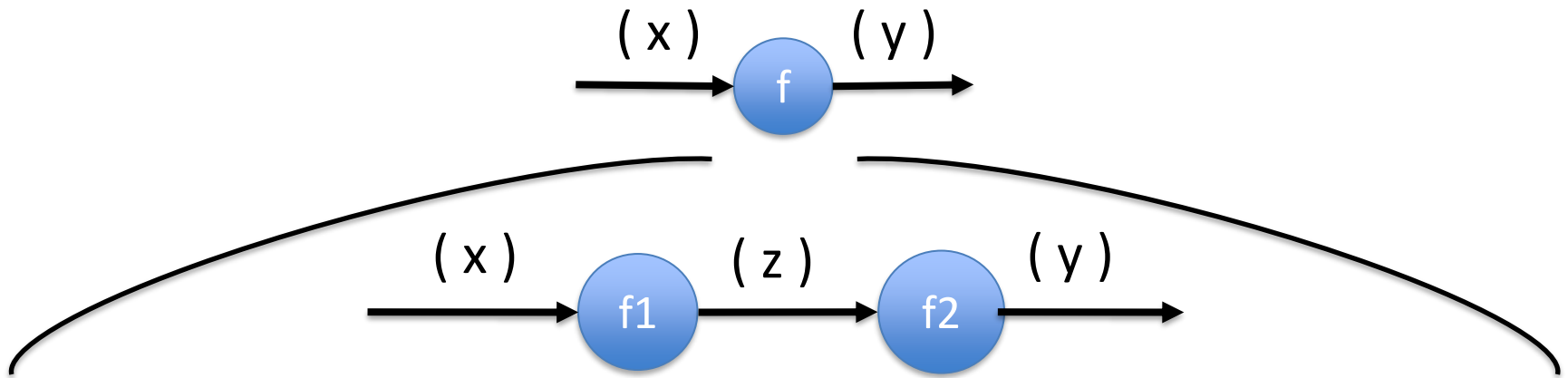


```
Y f(X x) {  
  return f1(x);  
}
```

```
Y f1(X x) {  
  var z = ....  
  
  return f2(z);  
}
```

```
Y f2(Z z) {  
  var y = ...  
  
  return y;  
}
```

Mit Abhängigkeiten (**FALSCH!!**)



```
Y f(X x) {  
  return f1(x);  
}
```

Integration

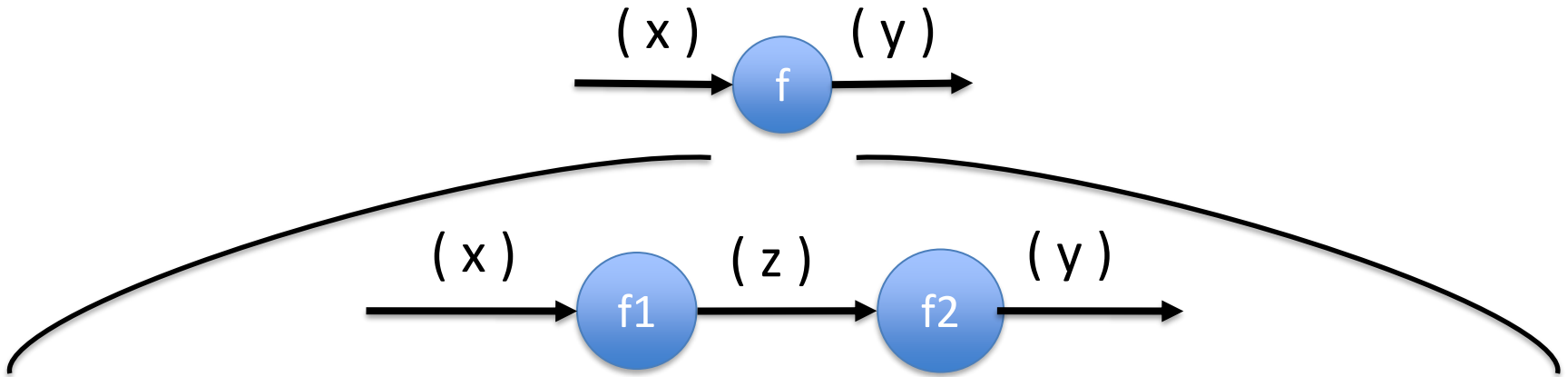
```
Y f1(X x) {  
  var z = ....  
  
  return f2(z);  
}
```

Integration +
Operation

```
Y f2(Z z) {  
  var y = ...  
  
  return y;  
}
```

Operation

Ohne Abhängigkeiten

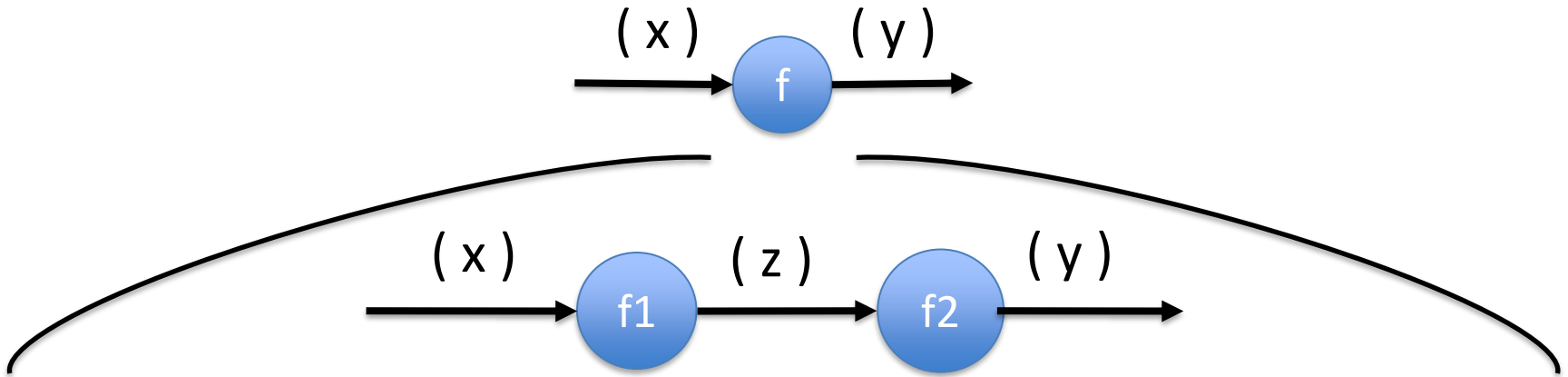


```
Y f(X x) {  
  var z = f1(x);  
  var y = f2(z);  
  return y;  
}
```

```
Z f1(X x) {  
  var z = ....  
  
  return z;  
}
```

```
Y f2(Z z) {  
  var y = ...  
  
  return y;  
}
```


Ohne Abhängigkeiten



```
Y f(X x) {  
  var z = f1(x);  
  var y = f2(z);  
  return y;  
}
```

Integration

```
Z f1(X x) {  
  var z = ....  
  
  return z;  
}
```

Operation

```
Y f2(Z z) {  
  var y = ...  
  
  return y;  
}
```

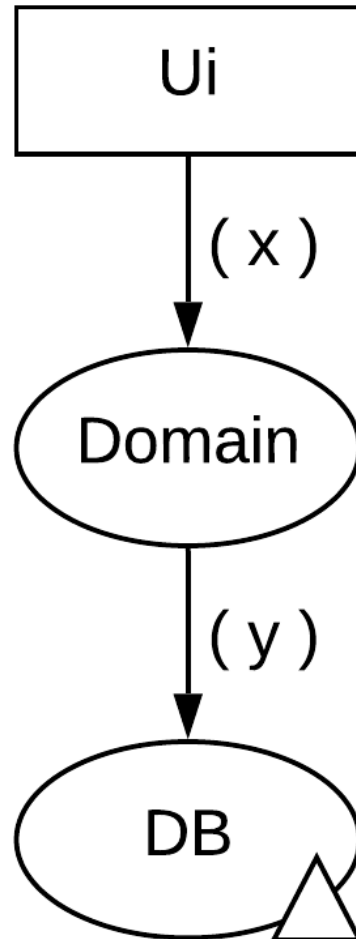
Operation

```
// "a=1;b=2;c=3" => {{ "a", "1"}, {"b", "2"}, {"c", "3"}}
```

```
public static IDictionary<string, string> ToDictionary(string configuration) {  
    var settings = SplitIntoSettings(configuration);  
    var keyValuePairs = SplitIntoKeyValuePairs(settings);  
    var dictionary = CreateDictionary(keyValuePairs);  
  
    return dictionary;  
}
```

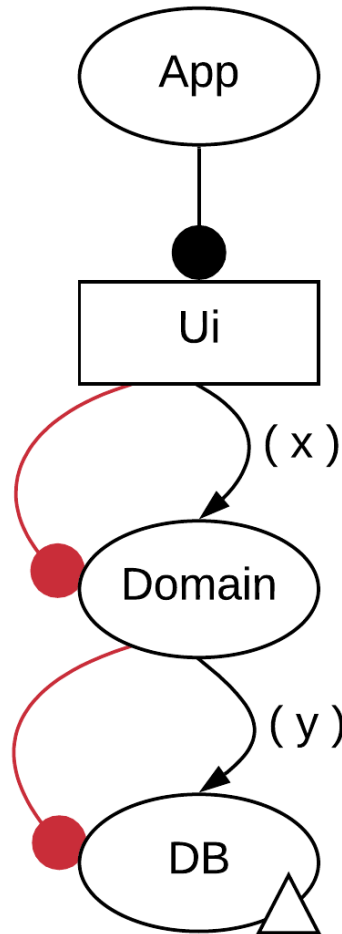
```
internal static IEnumerable<string> SplitIntoSettings(string configuration) {  
    return configuration.Split(';');  
}
```

Typischer Datenfluss



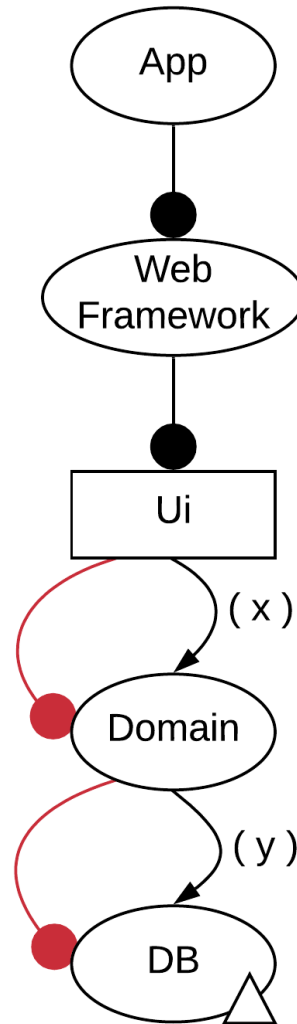
Typische Abhängigkeiten

Desktop



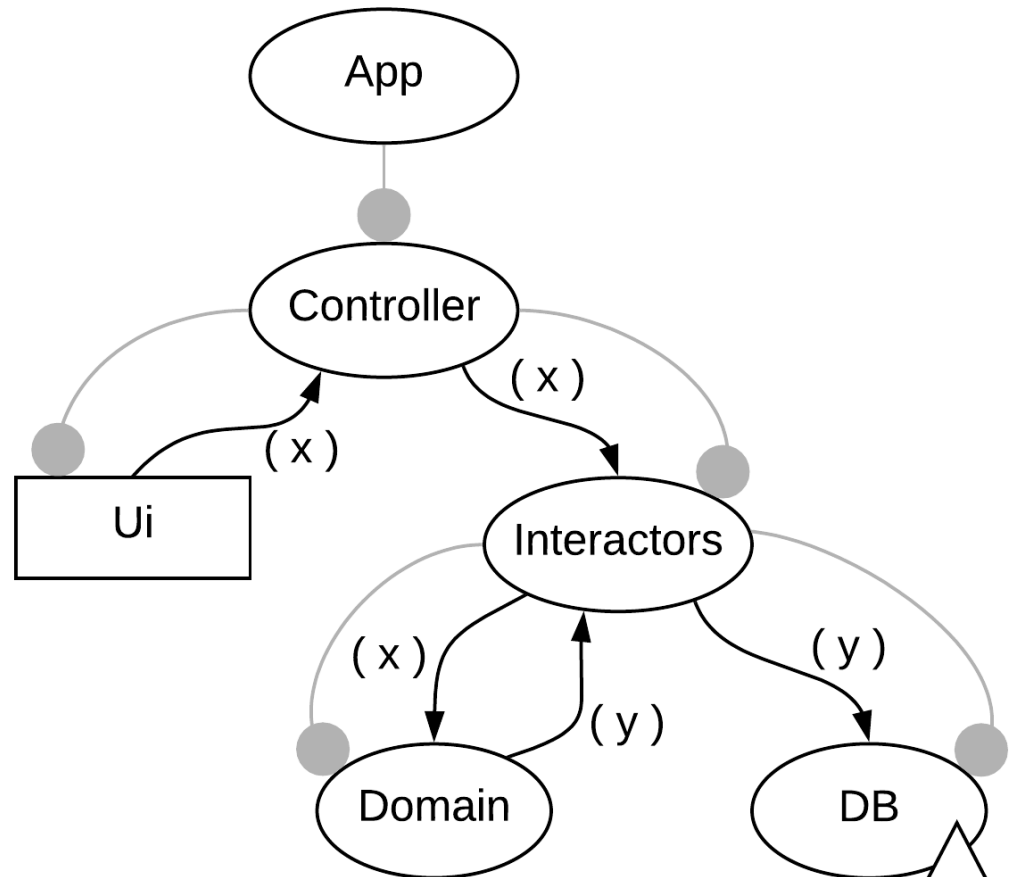
Typische Abhängigkeiten

Web



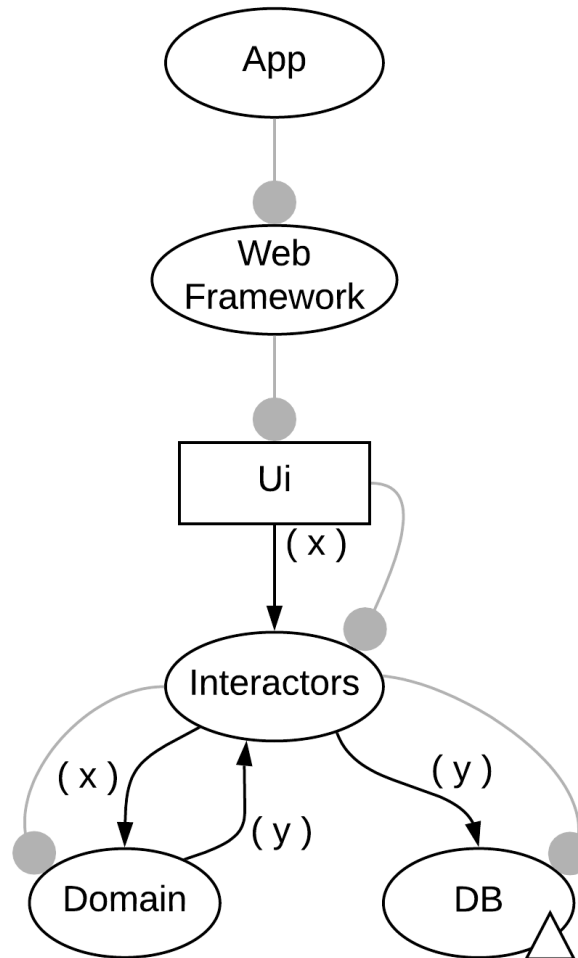
Korrekte Abhängigkeiten

Desktop

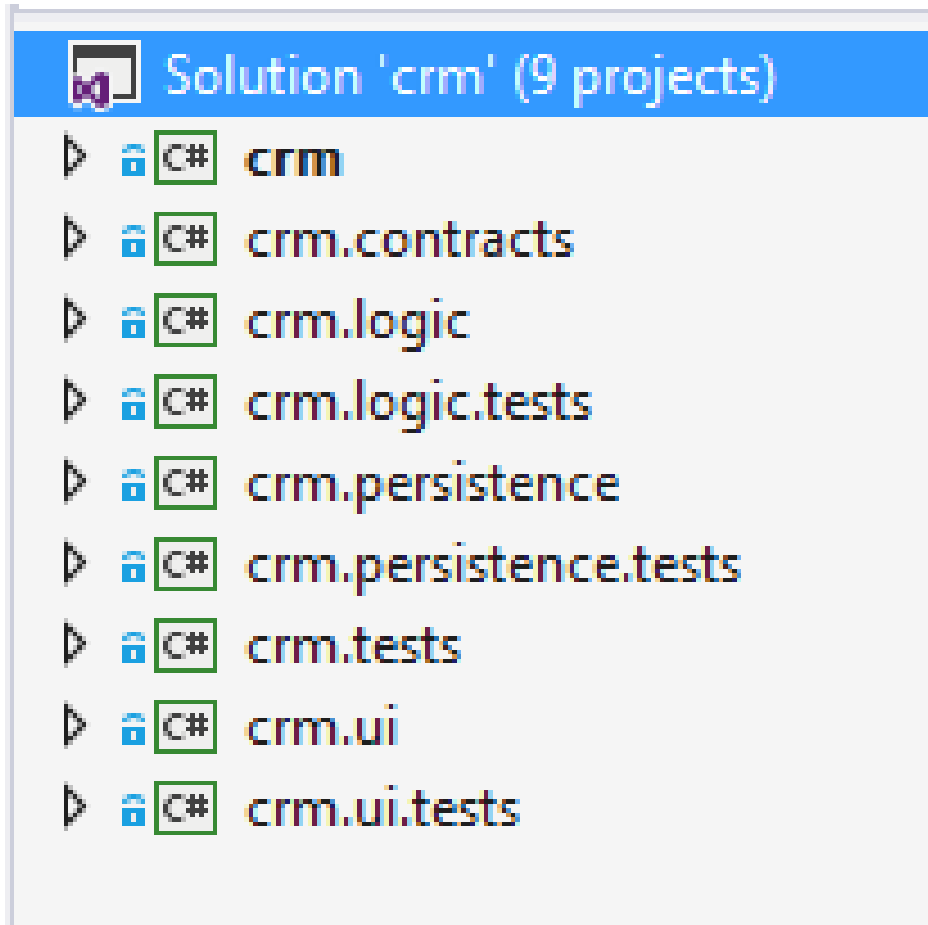


Korrekte Abhängigkeiten

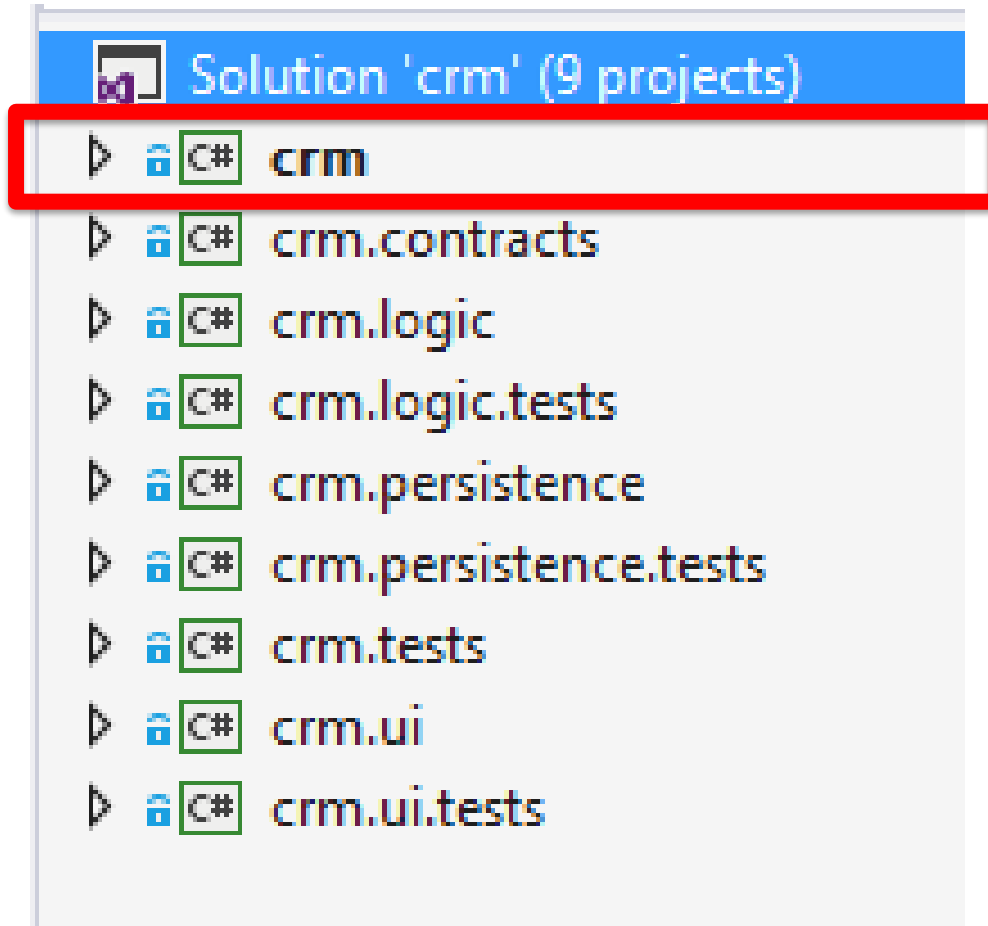
Web



Projektstruktur



Projektstruktur



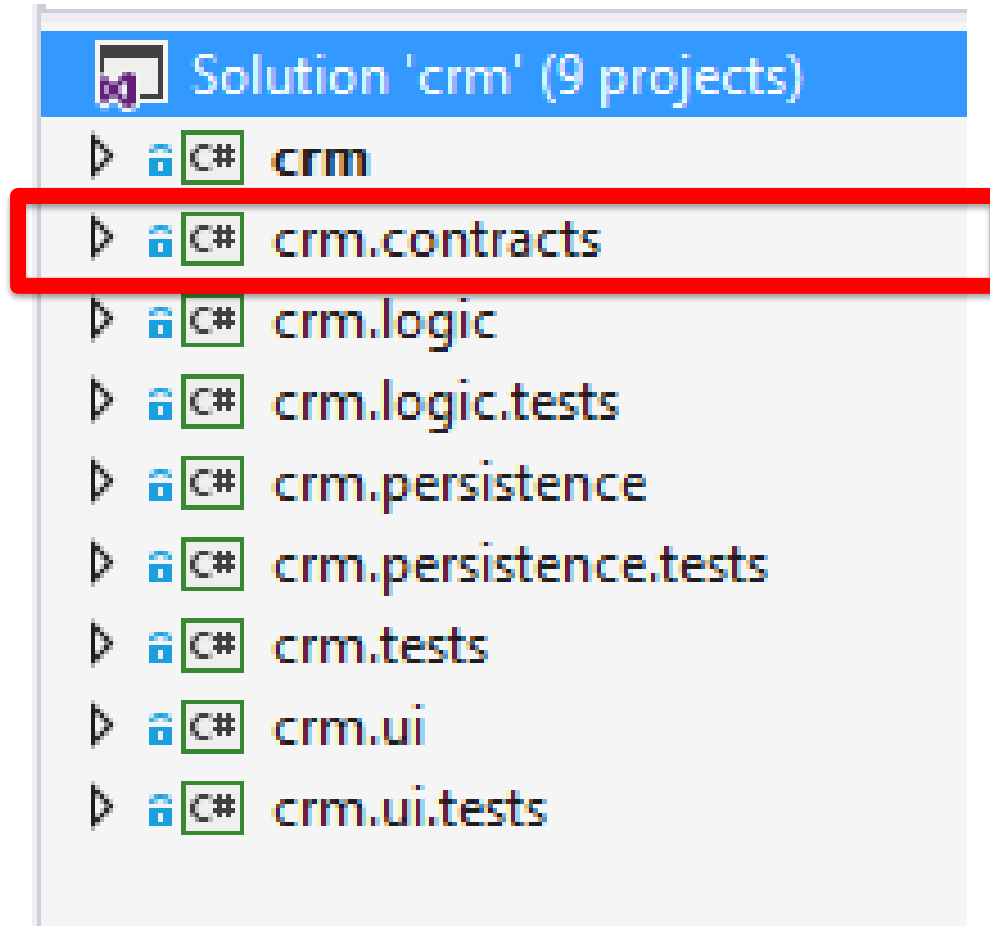
Application
Integration

.exe Projekt

maximale
Abhängigkeiten

Referenziert alle anderen
Projekte. Startet die
Anwendung in der Main
Methode.

Projektstruktur



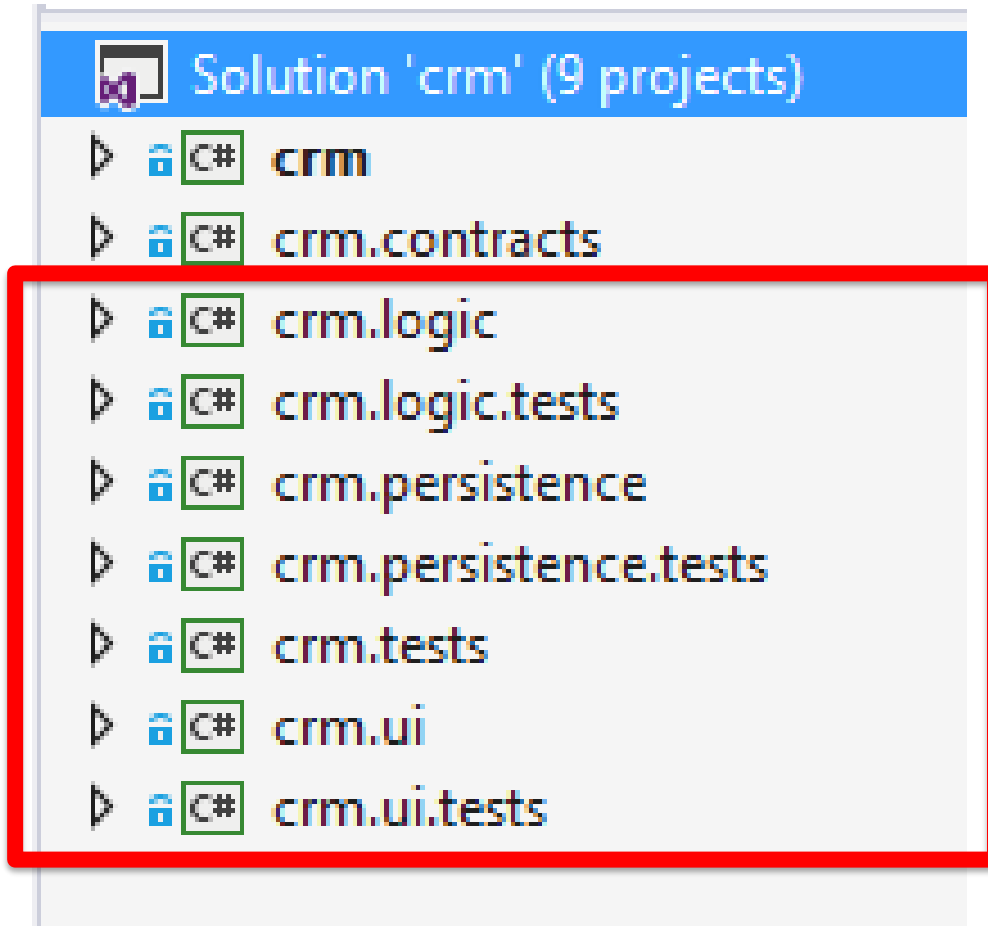
Kontrakte

Interfaces, Datentypen

.dll Projekt

keine Abhängigkeiten

Projektstruktur



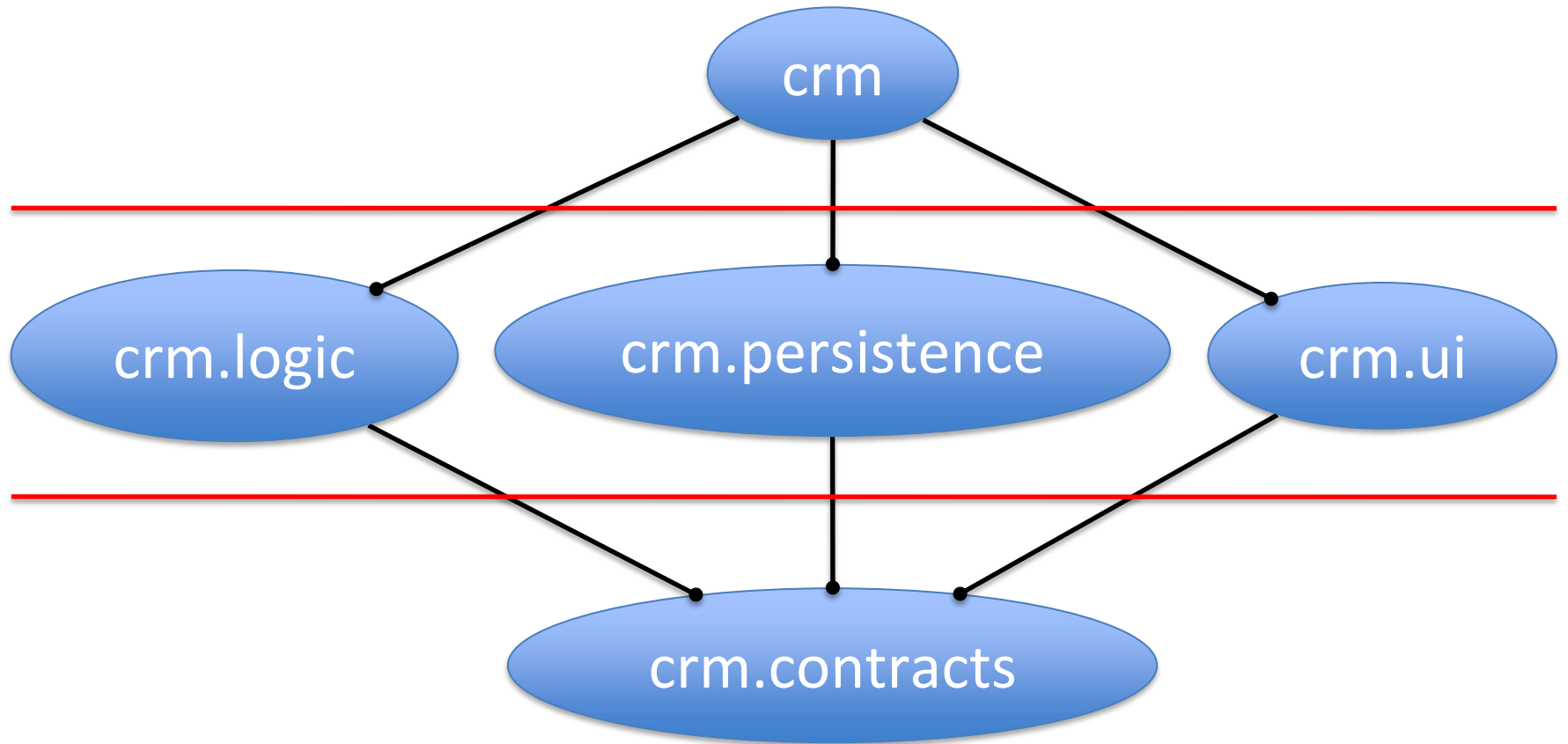
Komponenten

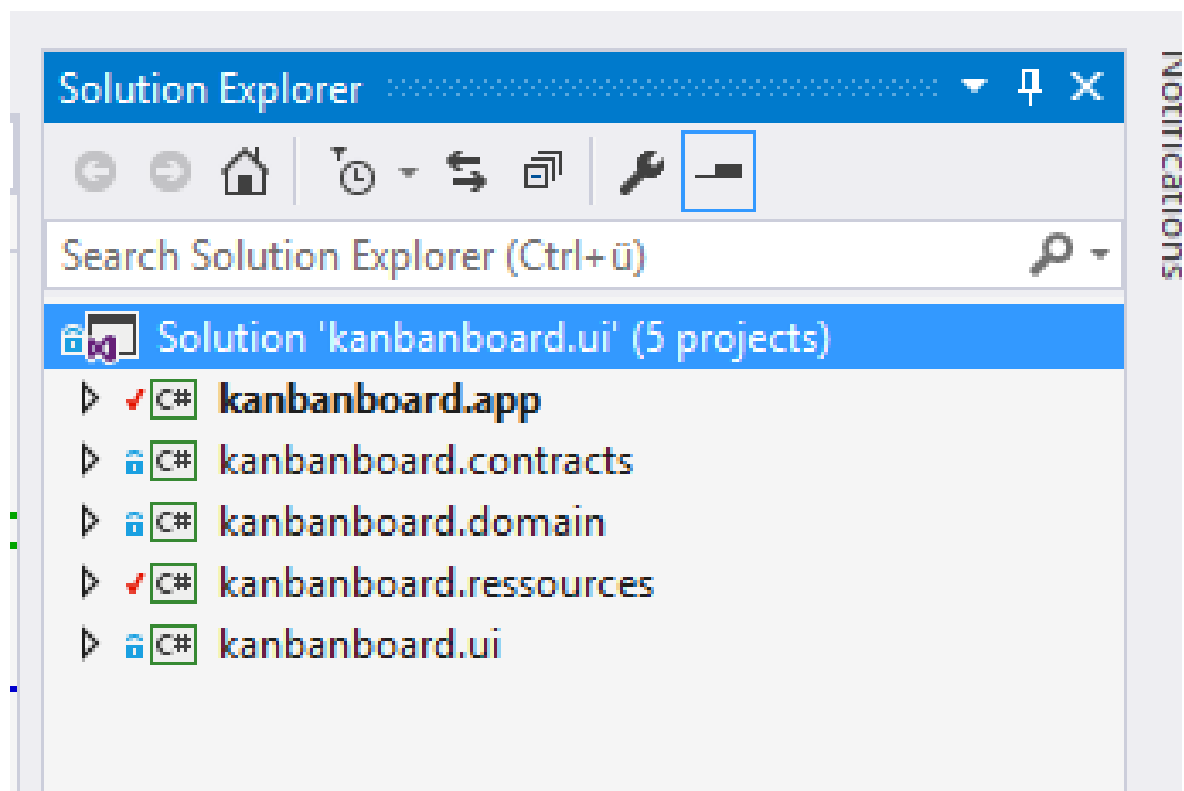
Logik, UI, Ressourcen,
etc.

.dll Projekte

Keine Abhängigkeiten,
referenziert lediglich
die Kontrakte.

Tests referenzieren
zugehörige Impl.



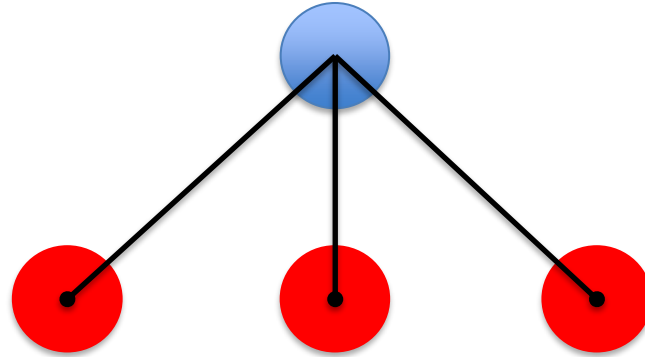


[STAThread]

```
public static void Main() {  
    var boardUi = new BoardUI();  
    var interactors = new Interactors();  
  
    // Start  
    var columns_and_tasks = interactors.Start();  
    boardUi.Setup_Board(columns_and_tasks);  
  
    // Create Task  
    boardUi.Create_Task += text => {  
        var task = interactors.Create_Task(text);  
        boardUi.Add_Task(task);  
    };  
  
    // Show Task  
    boardUi.Show_Task += (column, position) => {  
        var text = interactors.Show_Task(column, position);  
        boardUi.Display_Text(text);  
    };  
};
```

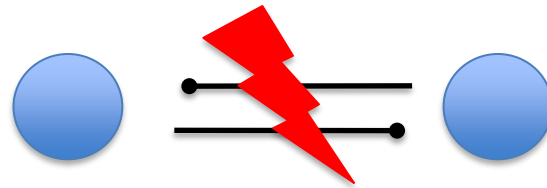
```
public Task Create_Task(string text) {  
    var column = kanban.Find_Column_for_new_Task();  
    var position = kanban.Find_Position_for_new_Task(column);  
    var task = kanban.Create_new_Task(text, column, position);  
    task = kanban.Add_new_Task(task);  
    task = kanban.Check_WIP_Limit(task);  
    task_Store.Save_Task(task);  
    notification.Notify_Clients();  
  
    return task;  
}
```

```
public void Save_Task(contracts.Task task) {  
    var dbTask = Map(task);  
    var collection = GetCollection();  
    collection.InsertMany(new [] { dbTask });  
}
```

Integration Operation
Segregation Principle (IOSP)

Principle of Mutual Oblivion (PoMO)



Prinzip der
gegenseitigen
Nichtbeachtung

Quellcode Beispiele

<https://github.com/slieser/flowdesignbuch>

Das nächste Webinar

30.06.2020 - 18:00

Flow Design am Beispiel

<https://stefanlieser.webinarninja.com/live-webinars/365899/register>



<https://flow-design.info/buch>



<http://refactoring-legacy-code.net>



<https://twitter.com/StefanLieser>



http://xing.com/profile/stefan_lieser



<http://linkedin.com/in/stefanlieser>